Springleik: A Case Study in Small Languages

Mark S. Williamsen

July 15, 2026

Abstract

Small languages arise in computing for a variety of reasons. A simple command interpreter can be useful in bringing up new hardware, particularly on small targets running embedded firmware. This leads naturally to a scriptable interface. Which can then evolve further into a primitive domain-specific language (DSL) that supports use cases in development, test, and even production. This work describes one path through the steps that lead from command interpreter to small language. Such languages can be very efficient with the resources they use including memory, processor bandwidth, and system services. Memory allocations can be static or dynamic, according to the details of the application. Additional features including self-documenting code and generation of baseline test outputs are easily implemented with minimal effort.

1 Model of software development

Many models are available for software development for this discussion. Here we present a simple model based on closed loop control theory, which includes concepts of time dependence and iteration. Figure 1 is a simulation diagram representing a software development process. The input on the left is requirements $R(s)$, and the output on the right is executable code and other deliverable assets $D(s)$. In between we have a means for creating code *Implement* positioned as the plant in control theory, and a means for testing code *Test* positioned as the sensor. The summing junction on the left compares test results $V(s)$ with requirements $R(s)$ to produce an error term $E(s)$, which then drives ongoing development. The directed edges connecting these parts together represent N-dimensional vectors where N is the number of requirements. The value of each dimension is the extent to which a given requirement has been met on some arbitrary scale.

A typical software project will begin with all vectors pointing to the origin: no requirements, no implementation, and no tests. As development proceeds we add requirements which drive the implementation, which is then tested and compared with requirements. Each directed edge conveys important information, but the most interesting vector is the error term $E(s)$. This will wander in an N-dimensional problem space, with N steadily increasing as development proceeds. Ideally this vector should always trend towards zero, with distance from the origin giving us an indication of how far we are from done. Time dependence is not shown explicitly but one can imagine one or more integrators

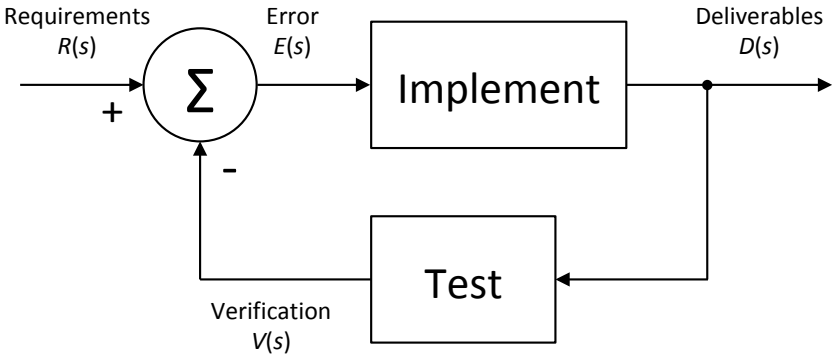

Figure 1: Software development process rendered as a closed-loop control simulation diagram. Negative feedback from the *Test* block causes the *Deliverables* to evolve, tracking changes in *Requirements*. When all requirements have been met, the error term $E(s)$ goes to zero and the control loop is in steady state.

in the loop, with time constants we may or may not control, modifying the time domain response of the development process.

Treating the development process as continuous we can write an expression for deliverables $D(s)$ in terms of implementation $I(s)$, test $T(s)$, and requirements $R(s)$, where s is the Laplace variable.

$$D(s) = R(s)\frac{I(s)}{1 + I(s)T(s)} \tag{1}$$

In this model we see that there are three things which matter: requirements, implementation, and tests. These pillars of software development should receive equal priority during development, so that all three can be considered as correct and complete when development ends and maintenance phase begins. As a practical matter however many projects end with one or more of the pillars incomplete. I claim without proof that given any one of these, the other two can be derived through various means. For the purposes of this paper we use the simulation diagram in figure 1 to define deriving implementation and tests from requirements as "forward engineering." Starting with an implementation or a test suite is then defined as "reverse engineering."

2 Object model for small targets

Developers of embedded software for small targets may be tempted to organize their object model around things the system has, such as inputs, outputs, sensors, and actuators. I would submit however that a coherent design is more likely to result from an object model organized around things the embedded system does, such as "home an axis", "measure a sample", "acquire a trace", or "energize a solenoid." The real reason we think in terms of objects is so that we can throw them into a container and iterate over them. While there

may be a use case for iterating over peripherals, the important use case here is iterating over a collection of nodes which represent actions a user might want to invoke. These are packaged along with the arguments needed to carry out each action. Thus we replace a noun-oriented design with a verb-oriented design so that command nodes can be collected into sequences which can then be interpreted on live hardware, simulated hardware, or a hybrid of both. Let's look at an example of how such a design might arise.

Consider a small target with embedded processor and several channels of analog-to-digital (ADC) conversion. On first bring-up we want to start characterizing the ADC inputs. We write a small monitor program with a command prompt exposed on a serial port, or a socket port if so equipped. Then we write commands to initialize the ADC, trigger a conversion, and report the result. Each of the commands can be followed with one or more arguments giving details such as sample rate and channel selection. Running a terminal program on a host computer allows us to manually exercise various behaviors of the ADC. We can stream a text file containing ADC commands from the host to the target's monitor program to execute a measurement sequence of arbitrary complexity. ADC conversions however take finite time, and we wind up overrunning the monitor program with new commands before old ones are complete. So we write a program to run on the host and carry out a series of ADC measurements on the target using monitor program commands, but now waiting for a prompt from the target before sending a new command from the host.

Many readers will have gone through these steps in bringing up a new target. We don't yet have a small language, but many of the pieces needed are already in place. Now we implement our object model, starting on the host side, declaring a class object for each command and an instance attribute for each argument the command needs. One can also add an instance attribute for the command's return value, if it makes sense for the particular use case. We can write a program on the host side which will interact with live hardware through the monitor command prompt, or with a simulation which emulates the same interface. Now we have a small language we can use to carry out the following steps:

- Instantiate one or more class objects on the host representing command nodes to be executed.
- Populate the nodes with arguments appropriate for the desired measurements.
- Compose the nodes into a program by appending them to a collection on the host.
- Iterate over the collection, interpreting each node as a command with arguments to be sent to the target.
- Capture results returned from the target for each command node, either as attributes of the command nodes or in a separate trace or listing.

Let's look at what we have. The class objects representing command nodes can live almost anywhere: on the host, on the target, in the cloud, or anywhere else that can interpret them into commands with the desired arguments. For

now we think of the arguments as being hard-coded, but they could just as well be derived or symbolic. The type of collection used to iterate over the command nodes doesn't matter much, except that iteration should follow a predictable repeatable path. The commands themselves can be sent to one or more target devices, or to any other device or simulation with an interface to receive them. Command replies (measurement results or error messages in our ADC example) can go back to the host, to some other monitor, or even be ignored. This then is the bare minimum of what's needed to claim we have a small domain-specific language. This can be prototyped and running in a matter of a few days, completely avoiding the trap that "nothing works until everything works."

Now let's look at what's missing. To obtain a text representation of the program we'll need a means to serialize the collection of command nodes. We have many well-established choices such as JSON, XML, YAML, etc. We also have the freedom to design our own representation if warranted, or dispense with source code altogether. In this paper we will assume without loss of generality that JSON has been chosen. Experience shows that it is trivial to give command node classes a serialize method which is specialized for specific commands and arguments. Instead of iterating over the collection while executing each node, we iterate over the collection while serializing each node to obtain a program listing. If we have multiple interpreters (host, target, cloud, etc.) all should produce an identical program listing for a given collection of command nodes. It should be mentioned here that while off-target interpreters can use the monitor command prompt interface, on-target interpreters don't have to since they can also call directly into functions on the target.

Real programming implies branching and looping, which is easily added by defining command nodes for conditional and repeated execution where the arguments represent loop count, limits for comparison, etc. At this point we've only described programs which are a simple list or sequence of nodes, so conditional and repeated execution will only affect nodes which are after the control node in the list. Nested iterations are implemented by appending more than one repeat node to the list. Multiple conditional nodes in the list must evaluate as "true" for nodes following them to be reached, a logical AND relationship. Everything we've done so far has a small footprint in terms of resources used on the target.

[Need either a visual diagram of node list, including repeats and branches, or pseudo code for the node list.]

An obvious choice for the collection container would be a linked list, with each node having a pointer or reference to the "next" node. If however one scans the list by calling node methods recursively, then one must be very aware of how much stack is used for each function call. I would urge readers to iterate without recursion whenever possible for this reason. If the list program being interpreted only changes at compile-time, then all nodes can be statically allocated. If the program may change at run-time then one must consider either a node pool or dynamic allocation. The linked list container is ideal for dynamic allocation and deallocation, since when you are done with the list you can iterate over it to release the memory used for each node.

The next step in our progression is to define list nodes so that they themselves can be lists of nodes, allowing our small language to represent lists of lists, which is to say composite tree structures. This incremental change, using all of the infrastructure we've developed so far, brings into focus a scalable,

portable, and verifiable framework for representing interpreted programs of arbitrary size, complexity, and capability. Now we have an abstract syntax tree (AST) interpreter optimized for the application at hand. The composite tree structure maps directly to structured text representations such as JSON, XML, or YAML. Human-readable program listings are easily obtained by serializing each node while traversing the tree with recursive descent. We encounter each node exactly once when we ask the root node to serialize the command tree. In this picture we can now say clearly that command nodes are dictionaries composed of key-value pairs representing all of the command arguments, along with an ordered list container holding pointers or references to subordinate nodes.

There are several features we take for granted in established programming languages which might not be needed in a small bespoke language. One is deserialization to parse a text representation into command nodes in memory. Another is the ability to parse and execute general math expressions. Experience shows that each of these features imposes a significant burden of time and expense to implement, document, and verify. Readers should consider just how much value would be added if they go down this path. Note that these features probably require two steps. First parsing the text representation into a syntax tree, and then rendering the syntax tree as command nodes that will be interpreted at run-time. We'll discuss deserialization more when we consider adding an analysis phase to our command tree structure.

Say something about rendering the tree in various human and/or machine readable forms. Ideally one-to-one mapping in all directions. With the possibility to run round-trip testing.

Say something about decorating the tree with measurement and analysis nodes. Which should lead naturally to a discussion of separation of execution and analysis.

Say something about machine-readable versus human-readable source code, bringing AI development and maintenance into the conversation.

Test and verification will lead naturally to my concept of an asymmetric tree comparison.